

Lập trình API - Bàn phím

Nguyễn Thu Hường

Khoa CNTT – Đại học GTVT

Các vấn đề của chương

- Những vấn đề cơ bản liên quan đến bàn phím
- Các thông điệp nhấn phím
- Các thông điệp ký tự
- Con chạy

Các thông điệp bàn phím

- Có 8 thông điệp khác nhau mà Windows sử dụng để cho biết các sự kiện từ bàn phím
  - Một chương trình có thể không cần quan tâm đến một nửa trong số chúng
  - Thông thường, những thông tin về bàn phím trong các thông điệp nhiều hơn những gì mà một chương trình cần
- Một chương trình không cần thiết phải xử lý tất cả các thông điệp mà nó nhận được, Windows tự nó đã xử lý nhiều chức năng của bàn phím
  - Ví dụ, chương trình có thể bỏ qua các tổ hợp phím chức năng hệ thống (chúng thường chứa phím **Alt**)

Tiêu điểm nhập (input focus)

- Khi một phím được nhấn, chỉ một cửa sổ nhận được thông điệp bàn phím
- Cửa sổ mà nhận được thông điệp bàn phím được gọi là cửa sổ đang sở hữu tiêu điểm nhập
- Cửa sổ đang có tiêu điểm nhập là:
  - Cửa sổ hiện hành (active window)
  - Hoặc là cửa sổ con cháu của cửa sổ hiện hành

Cửa sổ hiện hành

- Luôn luôn là cửa sổ ở mức đỉnh - là cửa sổ không có cha
  - Nếu cửa sổ hiện hành có thanh tiêu đề: thì Windows sẽ làm nổi thanh tiêu đề
  - Nếu cửa sổ hiện hành có khung hội thoại thay cho thanh tiêu đề: Windows sẽ làm nổi khung đó
  - Nếu cửa sổ hiện hành được thu nhỏ: Windows sẽ làm nổi nó trên thanh tác vụ

Tiêu điểm nhập

- Thông thường, các cửa sổ điều khiển cho biết chúng đang sở hữu tiêu điểm nhập bằng cách:
  - Nhấp nháy con chạy
  - Hoặc hiển thị một đường chấm chấm bao quanh
- Cũng có khi không có cửa sổ nào có tiêu điểm nhập cả
  - Đây là trường hợp khi mà tất cả các chương trình đang được thu nhỏ
  - Windows vẫn tiếp tục gửi các thông điệp bàn phím cho cửa sổ hiện hành, nhưng những thông điệp này có hình thức khác

Tiêu điểm nhập

- Một thủ tục cửa sổ có thể biết khi nào cửa sổ của nó nhận được tiêu điểm nhập bằng cách bắt các thông điệp WM_SETFOCUS và WM_KILLFOCUS
  - WM_SETFOCUS - cửa sổ đang nhận tiêu điểm nhập
  - WM_KILLFOCUS - cửa sổ đang bị mất tiêu điểm nhập

Hàng đợi và sự đồng bộ hoá

- Khi người sử dụng nhấn hoặc nhả các phím, các thông điệp bàn phím không được đưa vào hàng đợi thông điệp của chương trình ngay lập tức
  - Windows đưa những thông điệp này vào hàng đợi thông điệp hệ thống trước
  - Windows sẽ lấy thông điệp tiếp theo trong hàng đợi thông điệp hệ thống và đưa nó vào hàng đợi thông điệp của chương trình chỉ khi chương trình đã hoàn thành việc xử lý thông điệp trước

Hàng đợi và sự đồng bộ hoá

- Mục đích: giải quyết sự đồng bộ hoá
- Nguyên nhân:
  - Người sử dụng có thể thực hiện các thao tác gõ phím nhanh hơn là một chương trình có thể xử lý các lần nhấn phím
  - Một động tác nhấn phím đặc biệt có thể có hiệu lực chuyển đổi tiêu điểm nhập từ cửa sổ này sang cửa sổ khác

Thông điệp nhấn phím và thông điệp ký tự

- Các loại thông điệp bàn phím
  - Các thông điệp nhấn phím
  - Các thông điệp ký tự
- Đối với các tổ hợp nhấn phím mà chúng cho kết quả là các ký tự hiển thị, Windows sẽ gửi tới chương trình:
  - Các thông điệp nhấn phím
  - Và các thông điệp ký tự
- Ngược lại, Windows sẽ chỉ sinh ra các thông điệp nhấn phím

Các thông điệp nhấn phím

	Nhấn phím	Nhả phím
Thông điệp nhấn phím thường	WM_KEYDOWN	WM_KEYUP
Thông điệp nhấn phím hệ thống	WM_SYSKEYDOWN	WM_SYSKEYUP

- **Chú ý:** Nếu chúng ta giữ một phím ở trạng thái nhấn khá lâu thì Windows sẽ sinh ra:
  - Một chuỗi các thông điệp WM_KEYDOWN (hoặc WM_SYSKEYDOWN)
  - Và một thông điệp WM_KEYUP (hoặc WM_SYSKEYUP)

Các thông điệp nhấn phím hệ thống

- Các thông điệp WM_SYSKEYDOWN và WM_SYSKEYUP được sinh ra khi nhấn các phím kết hợp với phím Alt
  - Kích hoạt các mục trên thực đơn hoặc thực đơn hệ thống
  - Chuyển đổi cửa sổ hiện hành (Alt-Tab hoặc Alt-Esc)
  - Đóng một chương trình (Alt-F4)
  - Các chương trình thường bỏ qua các thông điệp WM_SYSKEYUP, WM_SYSKEYDOWN và chuyển chúng cho hàm DefWindowProc xử lý
- Chương trình cuối cùng cũng sẽ nhận được các thông điệp khác có liên quan đến kết quả của việc nhấn phím hệ thống (ví dụ như khi lựa chọn mục thực đơn)

Các thông điệp nhấn phím thường

- Các thông điệp WM_KEYDOWN và WM_KEYUP được sinh ra khi nhấn và nhả các phím mà không có phím Alt
  - Chương trình có thể sử dụng hoặc bỏ qua những thông điệp này
  - Còn Windows thì không quan tâm đến chúng

WM_KEYDOWN, WM_KEYUP, WM_SYSKEYDOWN, WM_SYSKEYUP

- Đối với cả 4 thông điệp trên:
  - wParam là mã phím ảo, nó xác định phím nào đang được nhấn hay nhả
  - lParam chứa một số dữ liệu khác liên quan đến việc nhấn phím

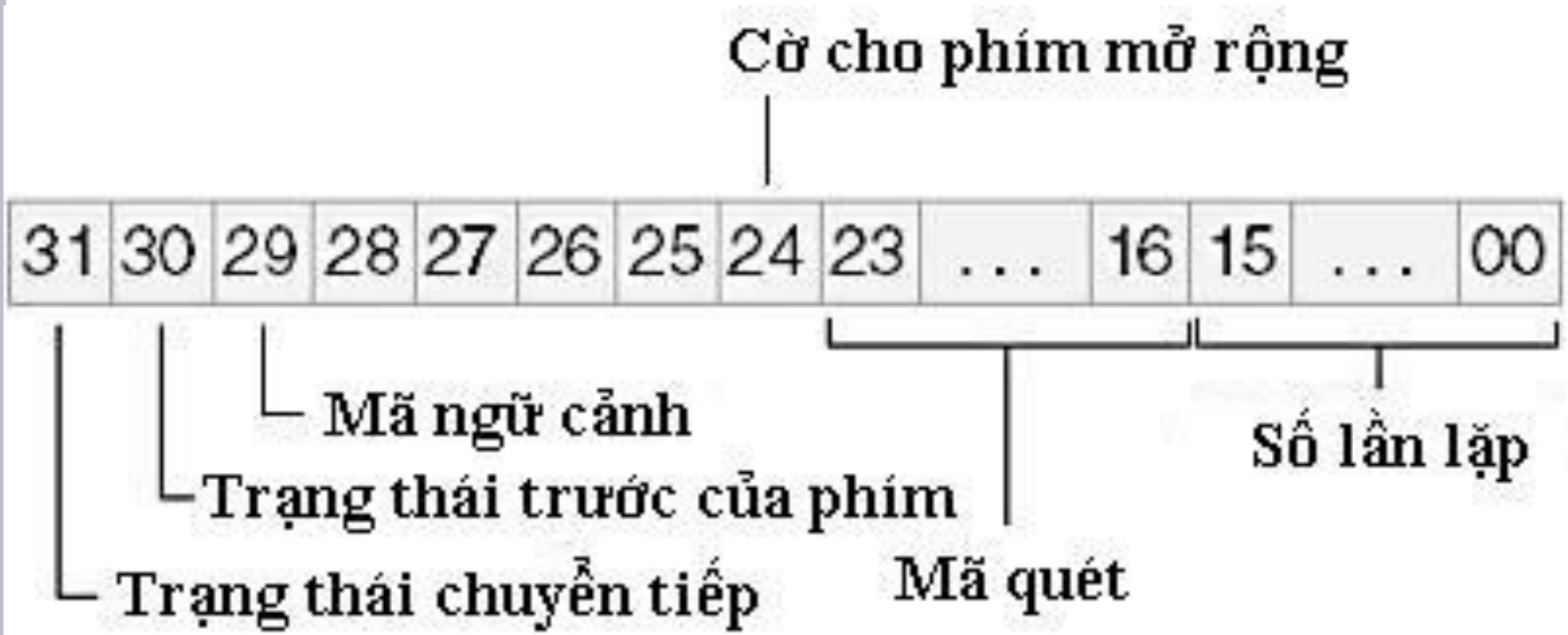
Các mã phím ảo

- Các mã phím **thực** là các mã mà được sinh ra bởi phần cứng của bàn phím vật lý - scan code (mã quét)
- Trên các bàn phím của hãng IBM
  - Mã quét 16 - phím Q
  - Mã quét 17 - phím W
  - Mã quét 18 - phím E
  - …
- Các mã quét
  - Phụ thuộc vào vị trí vật lý của các phím trên bàn phím
  - Phụ thuộc vào từng thiết bị (bàn phím) cụ thể

Các mã phím ảo

- Định nghĩa các mã phím ảo ⇒ Độc lập với thiết bị
- Một số mã phím ảo không thể được sinh ra trên các bàn phím của IBM
- Xem các trang 5, 6, 7, 8, 9

Tham số lParam của các thông điệp



Các trạng thái dịch chuyển

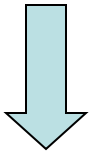
- Khi xử lý một thông điệp nhấn phím, có thể chúng ta cần biết có phím nào trong các phím dịch chuyển (Shift, Ctrl và Alt) hoặc các phím bật tắt (Caps Lock, Num Lock và Scroll Lock) đang được nhấn hay không
- Chúng ta có thể lấy được thông tin này bằng cách gọi hàm GetKeyState

Hàm GetKeyState

- Cách sử dụng
  - iState = GetKeyState (VK_SHIFT) ;
    - iState < 0 - phím Shift đang được nhấn
  - iState = GetKeyState (VK_CAPITAL) ;
    - iState có bit thấp nhất là 1 nếu phím Caps Lock đang được bật
- **Chú ý:** Hãy cẩn thận với hàm GetKeyState, bởi vì nó
  - **Không** thực hiện việc kiểm tra trạng thái của bàn phím **theo thời gian thực (lúc gọi hàm)**
  - Phản ánh trạng thái của bàn phím khi thông điệp được xử lý (**khi nhấn phím**)

Hàm GetKeyState

- Ví dụ, giả sử chúng ta cần phải dừng quá trình xử lý trong thủ tục cửa sổ cho đến khi người sử dụng nhấn phím chức năng F1:
  - while (GetKeyState (VK_F1) >= 0) ; // SAI !!!



Chương trình sẽ bị cheo (trừ khi thông điệp WM_KEYDOWN đối với phím F1 được lấy ra từ hàng đợi thông điệp trước khi thực hiện câu lệnh này)

- Nếu muốn biết trạng thái hiện thời thực của một phím, hãy sử dụng hàm GetAsyncKeyState

Sử dụng các thông điệp nhấn phím

- Hầu hết các chương trình **bỏ qua** tất cả các thông điệp nhấn phím
- Nếu xử lý các thông điệp WM_KEYDOWN, chúng ta có thể bỏ qua các thông điệp WM_KEYUP
- Các chương trình thông thường xử lý các thông điệp WM_KEYDOWN đối với các sự kiện nhấn phím không phải là các phím ký tự

Mở rộng chương trình DisplText1

- Xây dựng chương trình DisplText2 bằng cách mở rộng chương trình DisplText1 ở chương Hiển thị văn bản để có thể sử dụng các phím Page Up, Page Down, mũi tên lên và mũi tên xuống cho các thao tác cuốn văn bản theo chiều dọc

Các thông điệp ký tự


```
while (GetMessage (&msg, NULL, 0, 0))
{
    TranslateMessage (&msg) ;
    DispatchMessage (&msg) ;
}
```

- Hàm TranslateMessage có trách nhiệm
  - Chuyển đổi thông điệp nhấn phím thành thông điệp ký tự (nếu có thể)
  - Rồi đưa thông điệp ký tự này vào hàng đợi thông điệp (thông điệp ký tự này là thông điệp tiếp theo mà hàm GetMessage nhận được từ hàng đợi sau thông điệp nhấn phím)

Các thông điệp ký tự

	Các ký tự	Các ký tự chết
Các thông điệp ký tự thường	WM_CHAR	WM_DEADCHAR
Các thông điệp ký tự hệ thống	WM_SYSCHAR	WM_SYSDEADCHAR

- WM_CHAR và WM_DEADCHAR sinh ra từ WM_KEYDOWN
- WM_SYSCHAR và WM_SYSDEADCHAR sinh ra từ WM_SYSKEYDOWN
- Trong hầu hết các trường hợp, chương trình có thể chỉ xử lý thông điệp WM_CHAR và bỏ qua 3 thông điệp ký tự còn lại

Các thông điệp ký tự

	lParam	wParam
Các thông điệp nhấn phím		Mã phím ảo
Các thông điệp ký tự	Giống trên	Mã ký tự ANSI hoặc Unicode

Thứ tự của các thông điệp

- Các thông điệp ký tự được phân phát đến thủ tục cửa sổ của chương trình theo thứ tự nằm ở giữa các thông điệp nhấn phím
- Ví dụ, nếu phím Caps Lock không bật, và nhấn rồi nhả phím A, thủ tục cửa sổ sẽ nhận 3 thông điệp sau:

Thông điệp	Mã phím ảo (hoặc mã ký tự)
WM_KEYDOWN	Mã phím ảo của 'A' (0x41)
WM_CHAR	Mã ký tự của 'a' (0x61)
WM_KEYUP	Mã phím ảo của 'A' (0x41)

Thứ tự của các thông điệp

- Nếu gõ ký tự viết hoa A bằng cách nhấn phím Shift, nhấn phím A, nhả phím A, rồi nhả phím Shift, khi đó thủ tục cửa sổ sẽ nhận 5 thông điệp theo thứ tự sau:

Thông điệp	Mã phím ảo (hoặc mã ký tự)
WM_KEYDOWN	Mã phím ảo của VK_SHIFT (0x10)
WM_KEYDOWN	Mã phím ảo của 'A' (0x41)
WM_CHAR	Mã ký tự của 'A' (0x41)
WM_KEYUP	Mã phím ảo của 'A' (0x41)
WM_KEYUP	Mã phím ảo của VK_SHIFT (0x10)

Thứ tự của các thông điệp

- Nếu giữ lâu phím A ở trạng thái nhấn:

Thông điệp	Mã phím ảo (hoặc mã ký tự)
WM_KEYDOWN	Mã phím ảo của `A' (0x41)
WM_CHAR	Mã ký tự của `a' (0x61)
WM_KEYDOWN	Mã phím ảo của `A' (0x41)
WM_CHAR	Mã ký tự của `a' (0x61)
WM_KEYDOWN	Mã phím ảo của `A' (0x41)
WM_CHAR	Mã ký tự của `a' (0x61)
WM_KEYDOWN	Mã phím ảo của `A' (0x41)
WM_CHAR	Mã ký tự của `a' (0x61)
WM_KEYUP	Mã phím ảo của `A' (0x41)

Các ký tự điều khiển

- Phím Ctrl kết hợp với các phím chữ cái sinh ra các ký tự điều khiển ASCII từ 0x01 (Ctrl-A) đến 0x1A (Ctrl-Z)
- Một số mã ký tự điều khiển cũng được sinh ra bởi các phím sau:

Phím	Mã ký tự	Cũng được sinh ra bởi	Ký hiệu trong ngôn ngữ C
Backspace	0x08	Ctrl-H	\b
Tab	0x09	Ctrl-I	\t
Ctrl-Enter	0x0A	Ctrl-J	\n
Enter	0x0D	Ctrl-M	\r
Esc	0x1B	Ctrl-[	

Xử lý các ký tự điều khiển

- Các phím Tab, Enter, Backspace hoặc Escape sẽ sinh ra:
  - Các ký tự điều khiển ASCII (các thông điệp ký tự)
  - Và các mã phím ảo (các thông điệp nhấn phím)
- Những phím này nên được xử lý trong thông điệp WM_CHAR hay thông điệp WM_KEYDOWN?
- Sau nhiều năm suy nghĩ, tác giả dường như thích xử lý các phím Tab, Enter, Backspace, và Escape như các ký tự điều khiển (WM_CHAR) hơn là các phím ảo

Xử lý các ký tự điều khiển


```
case WM_CHAR:
    [các dòng lệnh khác]
    switch (wParam)
    {
    case `\b':          // backspace
        [các dòng lệnh khác]
        break ;
    case `\t':          // tab
        [các dòng lệnh khác]
        break ;
    case `\n':          // ký tự xuống dòng (linefeed)
        [các dòng lệnh khác]
        break ;
    case `\r':          // ký tự trở về đầu dòng (carriage return)
        [các dòng lệnh khác]
        break ;
    default:            // các mã ký tự
        [các dòng lệnh khác]
        break ;
    }
    return 0 ;
```

Các thông điệp ký tự chết

- Trên một số bàn phím không phải là các bàn phím tiếng Anh, một số phím được dùng để thêm các dấu phụ (như dấu trọng âm, dấu tách âm, …) cho các chữ cái
- Những phím này được gọi là những phím chết, bởi vì chúng không tự tạo ra các ký tự được

Các thông điệp ký tự chết

- Ví dụ, khi cài đặt bàn phím tiếng Đức, phím mà nằm cùng vị trí với phím +/= trên bàn phím U.S. (tiếng Mỹ) là phím chết, nó được dùng để thêm:
  - Dấu huyền (`) khi kết hợp với phím Shift
  - Hoặc dấu sắc (´) khi không nhấn phím Shift
- Khi người sử dụng nhấn phím chết này:
  - Thủ tục cửa sổ sẽ nhận được thông điệp WM_DEADCHAR với wParam bằng với mã ASCII hoặc Unicode của dấu phụ tương ứng
  - Sau đó, khi người sử dụng nhấn tiếp một phím chữ cái mà có thể được viết với dấu phụ này (ví dụ phím A), thủ tục cửa sổ sẽ nhận được thông điệp WM_CHAR với wParam là mã ANSI của chữ cái 'a' với dấu phụ đó

Chương trình KEYVIEW1

- Hiển thị bên trong miền thao tác của nó tất cả các thông điệp bàn phím mà nó nhận được (có tất cả 8 loại thông điệp ký tự)
- Giúp kiểm tra nội dung của bàn phím và các thông điệp ký tự mà Windows phân phát tới thủ tục cửa sổ của chương trình

Con chạy (caret)

- Khi chúng ta gõ văn bản trong một chương trình, con chạy cho biết ký tự tiếp theo mà chúng ta gõ sẽ xuất hiện ở đâu trên màn hình
- Các dạng phổ biến của con chạy
  - Dấu gạch dưới
  - Dấu thẳng đứng
  - Hộp nhỏ

Các hàm làm việc với con chạy

- Có 5 hàm quan trọng để làm việc với con chạy:
  - CreateCaret - Tạo con chạy gắn với một cửa sổ
  - SetCaretPos - Thiết lập vị trí của con chạy trong cửa sổ
  - ShowCaret - Hiển thị con chạy
  - HideCaret - Ẩn con chạy
  - DestroyCaret - Huỷ con chạy
- Ngoài ra còn:
  - GetCaretPos - lấy vị trí hiện thời của con chạy
  - GetCaretBlinkTime - lấy chu kỳ nhấp nháy con chạy
  - SetCaretBlinkTime - thiết lập chu kỳ nhấp nháy con chạy

Con chạy

- Theo thường lệ con chạy là:
  - Một dấu ngang hoặc một hộp nhỏ có kích thước bằng kích thước của một ký tự
  - Hoặc một dấu gạch đứng có chiều cao bằng chiều cao của một ký tự
- Chúng ta nên dùng dấu gạch đứng khi chúng ta sử dụng phông chữ tỷ lệ như phông chữ hệ thống mặc định của Windows

Nguyên tắc sử dụng con chạy

- Thủ tục cửa sổ
  - Gọi hàm CreateCaret trong khi xử lý thông điệp WM_SETFOCUS
  - Và gọi hàm DestroyCaret trong khi xử lý thông điệp WM_KILLFOCUS
- Sau khi gọi CreateCaret, thủ tục cửa sổ phải gọi ShowCaret để hiển thị con chạy
- Thủ tục cửa sổ
  - Phải ẩn con chạy nhờ hàm HideCaret mỗi khi nó thực hiện công việc vẽ trên cửa sổ trong khi xử lý thông điệp khác thông điệp WM_PAINT
  - Sau khi kết thúc công việc vẽ, chương trình cần gọi ShowCaret để hiển thị lại con chạy

Chương trình TYPER

- TYPER - một chương trình soạn thảo văn bản rất sơ đẳng. Người sử dụng có thể
  - Gõ văn bản trong cửa sổ
  - Di chuyển con chạy với các phím di chuyển con chạy
  - Xoá nội dung của cửa sổ bằng cách nhấn phím Escape
- Nội dung của cửa sổ cũng bị xoá khi bạn thay đổi kích thước cửa sổ hoặc thay đổi ngôn ngữ của bàn phím
- Ở đây không sử dụng các thanh cuốn, không có chức năng tìm kiếm, thay thế, lưu trữ tệp, …